COMPLETE TEST PLAN

Kicked Out of the Sky

Website & Merch Store

GitHub Pages (Frontend) + Vercel (Backend API)
Stripe (Payments) + Printful (Fulfillment)

Every test scenario needed for production readiness

How to Use This Document

This document contains every test you need to build before going to production. Each section is organized by which repo the tests belong in and what type of test they are.

- **Purple sections** = Backend repo (Vercel API) — use Jest
- **Green sections** = Frontend repo (GitHub Pages) — use Cypress
- **Amber sections** = Critical tests to prioritize first

The "What to Tell Copilot" column gives you the prompt language to use with GitHub Copilot to generate each test. Give Copilot your actual function code as context, then use the description from that column.

Work through the Critical priority tests first, then High, then Medium. You can ship to production once all Critical and High tests pass.

BACKEND REPO — UNIT TESTS (Jest)

1.1 Stripe Checkout Session Creation

Tests for the API endpoint that creates Stripe checkout sessions when a customer clicks "Buy".

#	Test Scenario	What to Tell Copilot	Priority
1	Valid request with correct product ID, variant, and quantity returns a Stripe session URL	*Write a Jest test that sends a valid checkout request and asserts the response contains a Stripe session URL*	**Critical**
2	Request with missing product ID returns 400 error	*Test that a checkout request without a product ID returns status 400 with an error message*	**Critical**
3	Request with invalid variant ID returns 400 error	*Test that a checkout request with a non-existent variant ID returns 400*	**High**
4	Request with zero or negative quantity returns 400 error	*Test that quantities of 0 and -1 both return 400 errors*	**High**
5	Request with non-integer quantity (e.g. 2.5) returns 400 error	*Test that a decimal quantity like 2.5 returns a 400 error*	**Medium**
6	Server-side price calculation matches expected price (never trust frontend price)	*Test that the function recalculates the price server-side from the product catalog and ignores any price sent from the frontend*	**Critical**
7	Price includes correct currency (e.g. USD)	*Test that the Stripe session is created with the correct currency code*	**High**
8	Checkout session includes correct success and cancel URLs	*Test that success_url and cancel_url in the Stripe session point to the correct pages*	**High**
9	Checkout session includes customer email if provided	*Test that when an email is included in the request, it is passed to Stripe as customer_email*	**Medium**
10	Request with excessively large quantity (e.g. 99999) is rejected or capped	*Test that an unreasonably large quantity is handled gracefully, either rejected or capped to a max*	**Medium**

1.2 Printful Order Creation

Tests for the function that creates a Printful order after payment is confirmed.

#	Test Scenario	What to Tell Copilot	Priority
1	Valid order data creates a Printful order with correct product, variant, and quantity	*Write a Jest test that calls the Printful order creation function with valid data and verifies the payload sent to Printful's API matches*	**Critical**
2	Shipping address is correctly formatted for Printful's API schema	*Test that the shipping address fields are mapped correctly to Printful's expected format (name, address1, city, state_code, country_code, zip)*	**Critical**
3	Order includes correct Printful variant ID (mapped from your product catalog)	*Test that your internal variant ID correctly maps to the Printful sync_variant_id*	**Critical**
4	Missing shipping address fields return an error before calling Printful	*Test that if address1, city, or zip are missing, the function throws an error without calling Printful*	**High**
5	International addresses are handled correctly (country codes, postal formats)	*Test that a non-US address with a valid country code is accepted and formatted correctly*	**High**
6	Printful API error response is caught and logged, not swallowed silently	*Mock Printful returning a 400 error and verify the function logs the error and throws a meaningful exception*	**Critical**
7	Printful rate limit (429) response triggers appropriate retry or error handling	*Mock a 429 response from Printful and test that the function either retries after a delay or returns a clear error*	**High**
8	Order metadata includes the Stripe payment/session ID for traceability	*Test that the Printful order includes external_id or metadata linking it back to the Stripe session*	**High**

1.3 Webhook Signature Verification

Tests for validating that incoming webhooks are genuinely from Stripe and Printful, not spoofed.

#	Test Scenario	What to Tell Copilot	Priority
1	Valid Stripe webhook signature passes verification	*Write a Jest test using stripe.webhooks.constructEvent with a valid test signature that passes without throwing*	**Critical**

#	Test Scenario	What to Tell Copilot	Priority
2	Invalid Stripe webhook signature is rejected with 400	*Test that a request with a tampered or missing stripe-signature header returns 400 and does not process the event*	**Critical**
3	Expired Stripe webhook (old timestamp) is rejected	*Test that a webhook with a timestamp outside the tolerance window is rejected*	**High**
4	Valid Printful webhook passes verification (if using webhook secret)	*Test that a Printful webhook with the correct secret/signature is accepted*	**High**
5	Invalid Printful webhook is rejected	*Test that a Printful webhook with a wrong or missing secret is rejected with 401 or 403*	**High**
6	Webhook with malformed JSON body returns 400	*Test that a webhook with unparseable JSON returns 400 without crashing the function*	**Medium**
7	Empty request body is rejected	*Test that an empty POST to the webhook endpoint returns 400*	**Medium**

1.4 Webhook Event Processing

Tests for the business logic that runs when valid webhook events are received.

#	Test Scenario	What to Tell Copilot	Priority
1	checkout.session.completed event triggers Printful order creation	*Test that receiving a checkout.session.completed event with valid session data calls your Printful order creation function with the correct arguments*	**Critical**
2	Duplicate checkout.session.completed events are handled idempotently (no duplicate orders)	*Test that processing the same session ID twice does not create two Printful orders — use a check for existing session ID*	**Critical**
3	payment_intent.payment_failed event is logged but does not create an order	*Test that a payment_failed event is logged and does not trigger order creation*	**High**
4	charge.refunded event triggers appropriate handling (log, notify, or cancel Printful order)	*Test that a charge.refunded event is processed and logs the refund; if you cancel the Printful order, verify that API call is made*	**High**
5	Unrecognized event types are acknowledged (200) but not processed	*Test that an unknown event type like customer.created returns 200 but triggers no business logic*	**Medium**
6	Printful order status webhook updates are logged	*Test that Printful status webhooks (shipped, returned, failed) are received and logged correctly*	**Medium**
7	If Printful order creation fails after successful payment, error is logged with enough context to manually fulfill	*Mock Printful returning 500 after a valid Stripe payment, and verify the error log includes the Stripe session ID, customer email, and product details*	**Critical**

1.5 Product & Variant Data

Tests for the endpoint(s) that serve product data to your frontend.

#	Test Scenario	What to Tell Copilot	Priority
1	Products endpoint returns all active products with correct fields (name, price, images, variants)	*Write a Jest test that calls the products endpoint and verifies the response contains the expected product structure*	**High**
2	Each product has at least one variant with a valid Printful sync_variant_id	*Test that every product returned has variants and each variant has a valid sync_variant_id*	**High**
3	Product prices are returned in the smallest currency unit (cents for USD)	*Test that prices are integers in cents, not floating point dollars*	**High**
4	Out-of-stock or discontinued variants are either excluded or flagged	*Test that a variant marked as discontinued in Printful is either not returned or has an availability flag set to false*	**Medium**
5	Product images are valid URLs that resolve	*Test that image URLs in product data are valid HTTPS URLs*	**Medium**
6	Products endpoint handles Printful API being down gracefully (cached fallback or clear error)	*Mock Printful returning 500 and verify the products endpoint either serves cached data or returns a clear error, not a crash*	**High**

1.6 Shipping Rate Calculation

Tests for shipping rate estimation if you provide this before checkout.

#	Test Scenario	What to Tell Copilot	Priority
1	Valid US address returns shipping rates from Printful	*Test that a valid US shipping address returns at least one shipping rate with a method name and cost*	**High**
2	Valid international address returns shipping rates	*Test that a valid international address (e.g. Portugal) returns shipping rates*	**High**
3	Invalid or incomplete address returns a clear error	*Test that a missing zip code returns a clear error message, not a Printful API error dump*	**Medium**
4	Shipping rates are returned in the correct currency	*Test that shipping rate amounts match the expected currency*	**Medium**

1.7 Input Validation & Security

Tests for request validation, sanitization, and security hardening across all endpoints.

#	Test Scenario	What to Tell Copilot	Priority
1	All endpoints reject non-POST requests (GET, PUT, DELETE) with 405	*Test that GET, PUT, and DELETE requests to your checkout and webhook endpoints return 405 Method Not Allowed*	**High**
2	SQL injection / XSS in input fields is sanitized or rejected	*Test that sending script tags or SQL injection strings in address fields does not cause errors and the input is sanitized*	**High**
3	Extremely long string inputs are rejected (field length limits)	*Test that a name or address field with 10,000 characters is rejected with a 400 error*	**Medium**
4	CORS headers are set correctly (only your GitHub Pages domain is allowed)	*Test that the CORS Access-Control-Allow-Origin header matches your GitHub Pages URL and rejects other origins*	**Critical**
5	API rate limiting prevents abuse (if implemented)	*Test that sending 100 rapid requests to the checkout endpoint triggers rate limiting after the threshold*	**Medium**
6	Environment variables are not exposed in error responses	*Test that error responses do not contain API keys, secrets, or environment variable values*	**Critical**

BACKEND REPO — INTEGRATION TESTS (Jest)

2.1 Stripe Integration (Test Mode)

These tests use real Stripe test API keys to verify the connection works end-to-end.

#	Test Scenario	What to Tell Copilot	Priority
1	Create a real checkout session using Stripe test keys and verify a valid session ID is returned	*Write an integration test that creates a Stripe checkout session with test keys and asserts a valid session ID starting with cs_ is returned*	**Critical**
2	Retrieve the created session and verify line items match what was sent	*After creating a session, retrieve it via stripe.checkout.sessions.retrieve and verify the line items match*	**High**
3	Verify the session contains correct metadata (order ID, product details)	*Test that the session metadata includes your custom fields for order tracking*	**High**
4	Test with Stripe's special test card numbers for specific scenarios (decline, insufficient funds)	*Document which Stripe test card numbers to use: 4242424242424242 (success), 4000000000000002 (decline), 4000000000009995 (insufficient funds)*	**High**

2.2 Printful Integration (Test Mode)

These tests use Printful test/sandbox credentials to verify order creation works.

#	Test Scenario	What to Tell Copilot	Priority
1	Create a test order in Printful and verify it returns a valid order ID	*Write an integration test that creates a Printful order using test credentials and asserts a valid order ID is returned*	**Critical**
2	Verify the test order contains the correct product, variant, and shipping address	*After creating the order, retrieve it from Printful and verify all fields match what was sent*	**High**
3	Fetch product catalog from Printful and verify your products exist with correct variant IDs	*Test that your synced products are accessible via the Printful API and variant IDs match your internal catalog*	**High**

#	Test Scenario	What to Tell Copilot	Priority
4	Shipping rate estimation returns valid rates for a test address	*Call Printful's shipping rate endpoint with a test address and verify rates are returned*	**Medium**

2.3 Stripe → Printful Pipeline

The most critical integration: verifying the full payment-to-fulfillment pipeline.

#	Test Scenario	What to Tell Copilot	Priority
1	Simulated checkout.session.completed webhook creates a Printful order with correct details	*Write an integration test that sends a Stripe webhook event (checkout.session.completed) to your webhook endpoint and verifies a Printful order is created via the Printful API*	**Critical**
2	Printful order failure after Stripe success: verify error logging contains all info needed for manual fulfillment	*Mock Printful returning 500, send a valid Stripe webhook, and verify logs contain session ID, email, product, variant, address*	**Critical**
3	Concurrent webhook deliveries for the same session are handled without duplicate orders	*Send the same checkout.session.completed event twice simultaneously and verify only one Printful order is created*	**High**

FRONTEND REPO — CYPRESS E2E TESTS

3.1 Product Browsing & Display

Tests that verify products load and display correctly for customers.

#	Test Scenario	What to Tell Copilot	Priority
1	Merch page loads and displays all products	*Write a Cypress test that visits the merch page and asserts all expected products are visible with names and images*	**Critical**
2	Each product shows the correct price	*Test that each product's displayed price matches the expected value (formatted with $ and two decimals)*	**Critical**
3	Product images load without broken image icons	*Test that all product images have naturalWidth > 0 (loaded successfully) and no broken image placeholders*	**High**
4	Clicking a product shows its detail view with all variants (sizes, colors)	*Test that clicking a product opens a detail view showing all available size and color options*	**High**
5	Selecting a variant updates the displayed price if variants have different prices	*Test that switching between variants updates the displayed price*	**Medium**
6	Product descriptions render correctly (no raw HTML or broken formatting)	*Test that product descriptions are visible and contain no raw HTML tags*	**Medium**
7	Products page handles API error gracefully (shows error message, not blank page)	*Use cy.intercept to mock a 500 response from your products API and verify the page shows an error message*	**High**
8	Products page shows loading state while fetching	*Use cy.intercept with a delay and verify a loading indicator appears while products are being fetched*	**Medium**

3.2 Cart Functionality

Tests for the shopping cart experience.

#	Test Scenario	What to Tell Copilot	Priority
1	Adding a product to cart updates the cart count	*Write a Cypress test that adds a product to the cart and verifies the cart icon/ count increments*	**Critical**
2	Adding the same product twice increases quantity, not duplicate entries	*Test that adding the same variant twice results in quantity 2, not two separate line items*	**High**
3	Removing an item from cart updates the total and count	*Test that removing an item from the cart decreases the total and cart count correctly*	**High**
4	Cart persists across page navigation (if using localStorage or session)	*Test that after adding items and navigating to another page and back, the cart still contains the items*	**High**
5	Changing quantity in cart updates the line item total and cart total	*Test that changing an item's quantity from 1 to 3 updates both the line total and cart total*	**High**
6	Cart total calculation is correct with multiple different products	*Add three different products with different prices and quantities, then verify the cart total matches the expected sum*	**Critical**
7	Empty cart shows appropriate message and disabled checkout button	*Test that an empty cart displays a message like 'Your cart is empty' and the checkout button is disabled or hidden*	**Medium**
8	Cart handles unavailable variants gracefully (if product removed from Printful)	*Mock the products API to return one fewer variant, and test that the cart handles a now-invalid item gracefully*	**Medium**

3.3 Checkout Flow — Happy Path

The most important E2E test: a customer successfully buying merch.

#	Test Scenario	What to Tell Copilot	Priority
1	Full happy path: browse → select product → pick variant → add to cart → enter shipping → proceed to Stripe → see confirmation	*Write a Cypress E2E test for the complete purchase flow: visit merch page, click a product, select a size, add to cart, enter shipping details, click checkout, and verify redirect to Stripe or confirmation page*	**Critical**

#	Test Scenario	What to Tell Copilot	Priority
2	Checkout button sends correct data to your backend API	*Use cy.intercept to spy on the checkout API call and verify the request body contains the correct product ID, variant ID, quantity, and shipping address*	**Critical**
3	Successful checkout redirects to the success/ confirmation page	*After a successful checkout API response (mock with cy.intercept if needed), verify the user is redirected to the success page*	**Critical**
4	Success page displays order confirmation details	*Test that the success page shows a confirmation message and relevant order details*	**High**
5	Cancel/back from Stripe returns to cart with items preserved	*Test that the cancel URL from Stripe returns the user to the cart page with their items still intact*	**High**

3.4 Checkout Flow — Error Handling

Tests for what customers see when things go wrong.

#	Test Scenario	What to Tell Copilot	Priority
1	Backend API returning 500 shows a user-friendly error (not a technical error dump)	*Use cy.intercept to mock a 500 from your checkout API and verify the user sees a friendly error message like 'Something went wrong, please try again'*	**Critical**
2	Backend API timeout shows a timeout error message	*Use cy.intercept with a long delay to simulate a timeout and verify the user sees an appropriate message*	**High**
3	Network failure (offline) shows a connection error	*Use cy.intercept to force a network error and verify the user sees a connection error message*	**High**
4	Double-click on checkout button does not submit twice	*Use cy.intercept to spy on checkout API calls, click the checkout button rapidly twice, and verify only one request was made*	**Critical**
5	Checkout button shows loading state while processing	*Test that the checkout button shows a spinner or 'Processing...' text and is disabled while the API call is in flight*	**High**
6	Stripe redirect failure is handled (e.g. Stripe.js fails to load)	*Mock Stripe.js failing to redirect and verify the user sees an error instead of a blank page*	**Medium**

3.5 Shipping & Address Form

Tests for the shipping address form and validation.

#	Test Scenario	What to Tell Copilot	Priority
1	All required fields show validation errors when submitted empty	*Write a Cypress test that submits the shipping form with all fields empty and verifies each required field shows an error*	**High**
2	Invalid email format shows validation error	*Test that entering 'notanemail' in the email field shows an email validation error*	**High**

#	Test Scenario	What to Tell Copilot	Priority
3	Valid US address is accepted	*Test that a complete valid US address passes validation*	**High**
4	Valid international address is accepted (e.g. Portugal)	*Test that a complete valid Portuguese address with the correct country selected passes validation*	**High**
5	State/province field adapts to selected country	*Test that changing the country updates the state/province dropdown or field appropriately*	**Medium**
6	Zip/postal code format validation matches selected country	*Test that US zip requires 5 digits and PT postal code accepts the correct format*	**Medium**
7	Special characters in address fields are handled (accents, non-Latin characters)	*Test that addresses with accented characters (like Portuguese names) are accepted without errors*	**Medium**

3.6 Responsive & Cross-Browser

Tests to ensure the site works on your fans' actual devices.

#	Test Scenario	What to Tell Copilot	Priority
1	Merch page renders correctly at mobile viewport (375px width)	*Set cy.viewport(375, 667) and verify the merch page layout is not broken — products are visible and no horizontal scroll*	**Critical**
2	Cart is accessible and functional on mobile	*At mobile viewport, test that the cart can be opened, items viewed, and quantities changed*	**High**
3	Checkout form is usable on mobile (fields not cut off, keyboard doesn't obscure submit button)	*At mobile viewport, test that all shipping form fields are visible and the submit button is reachable*	**High**
4	Navigation menu works on mobile (hamburger menu if applicable)	*At mobile viewport, test that the navigation menu can be opened and links work*	**High**
5	Product images scale correctly on tablet (768px)	*Set cy.viewport(768, 1024) and verify product images scale proportionally without distortion*	**Medium**
6	Desktop layout displays correctly (1280px+)	*At desktop viewport, verify the full layout renders with proper grid/columns*	**Medium**

3.7 Site Navigation & General UX

Tests for the overall site experience beyond the merch store.

#	Test Scenario	What to Tell Copilot	Priority
1	Home page loads without errors	*Write a Cypress test that visits the home page and asserts no console errors and key elements are visible*	**High**
2	All navigation links work and lead to correct pages	*Test that each nav link navigates to the expected URL and the page loads*	**High**
3	404 page displays for invalid URLs	*Test that visiting a non-existent URL shows your 404 page (note: GitHub Pages has specific 404 handling)*	**Medium**
4	External links (streaming platforms, social media) open correctly	*Test that links to Spotify, Apple Music, Instagram etc. have correct href values and target=_blank*	**Medium**
5	Page load performance is acceptable (no massive blocking resources)	*Use cy.lighthouse or just verify the page becomes interactive within a reasonable time*	**Medium**
6	Meta tags and SEO basics are present (title, description, OG tags)	*Test that the page has a title tag, meta description, and Open Graph tags for social sharing*	**Medium**

MANUAL TESTING CHECKLIST — Before Launch

Some things are hard or impossible to automate. Do these manually before going live.

4.1 Full Purchase Test (Test Mode)

- Place a complete test order using Stripe test card 4242 4242 4242 4242
- Verify the order appears in your Stripe test dashboard
- Verify the order appears in Printful's test dashboard
- Verify the order details match (product, variant, size, quantity, address)
- Verify the customer would receive a confirmation (email or on-screen)
- Test a declined card (4000 0000 0000 0002) and verify the error message is helpful
- Test the cancel flow — click back from Stripe checkout and verify cart is intact

4.2 Real Device Testing

- Test on your iPhone (Safari) — complete a full browse + checkout flow
- Test on an Android device if possible (Chrome)
- Test on desktop Chrome, Firefox, and Safari
- Verify images load quickly and are not blurry
- Verify text is readable at all screen sizes
- Verify buttons are tap-friendly on mobile (minimum 44px touch target)

4.3 Content & Visual Review

- Proofread all product names, descriptions, and prices
- Verify all product images are the correct ones for each variant
- Check that size charts or variant labels are accurate
- Verify the band name, logo, and branding are consistent across all pages
- Check the success page message — does it make sense and feel on-brand?
- Verify error messages are friendly and don't expose technical details

4.4 Pre-Launch Environment Checks

- Stripe API keys are switched from test to live (and test keys are removed from production)
- Printful is connected with live credentials, not test/sandbox
- Webhook endpoints are configured in Stripe dashboard for production URL
- Webhook endpoints are configured in Printful dashboard for production URL

- CORS settings on Vercel only allow your actual GitHub Pages domain
- Environment variables on Vercel are set for production (not test keys)
- SSL/HTTPS is working on both frontend and backend
- Custom domain (if any) is configured and DNS is propagated
- Error monitoring (Sentry or similar) is configured and receiving test events
- Vercel function logs are accessible and you know where to find them

4.5 Post-Launch Smoke Test

- Place one real order with a real card immediately after launch (buy your own merch!)
- Verify the charge appears in Stripe live dashboard
- Verify the order appears in Printful live dashboard
- Verify you receive any confirmation/notification emails
- Monitor Vercel function logs for the first few hours for any errors
- Check error monitoring dashboard for any unhandled exceptions

CI/CD PIPELINE SETUP

5.1 Backend Repo — GitHub Actions

Tell Copilot to create a GitHub Actions workflow file at .github/workflows/test.yml that does the following:

- Triggers on every push and pull request to main
- Sets up Node.js (match your Vercel runtime version)
- Runs npm install
- Runs npm test (Jest unit tests)
- Optionally runs integration tests with Stripe test keys (stored as GitHub secrets)
- Fails the build if any test fails
- Reports test results in the PR checks

5.2 Frontend Repo — GitHub Actions

Tell Copilot to create a GitHub Actions workflow for Cypress:

- Triggers on every push and pull request to main
- Sets up Node.js
- Runs npm install
- Starts a local static server for your frontend (e.g. npx serve)
- Runs Cypress tests against the local server
- Configures the backend URL as an environment variable pointing to your Vercel staging/preview
- Uploads Cypress screenshots and videos as GitHub Actions artifacts on failure
- Fails the build if any Cypress test fails

5.3 Test Summary

Category	Repo	Tool	Test Count	Priority
Unit Tests	Backend	Jest	42	Critical/High
Integration Tests	Backend	Jest	11	Critical/High
E2E Tests	Frontend	Cypress	37	Critical/High
Manual Tests	Both	Manual	26	All Critical
CI/CD	Both	GitHub Actions	2 workflows	High

TOTAL			116+	

Start with all Critical tests across both repos. Once those pass, work through High priority. You can launch with all Critical and High tests passing — Medium priority tests can be added after launch to harden things further.